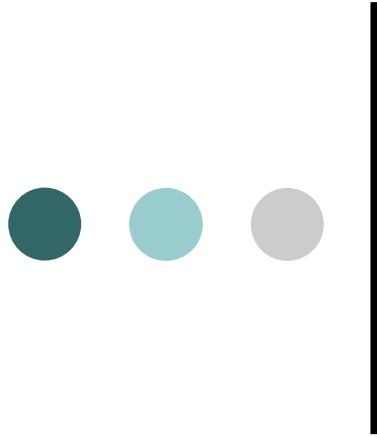
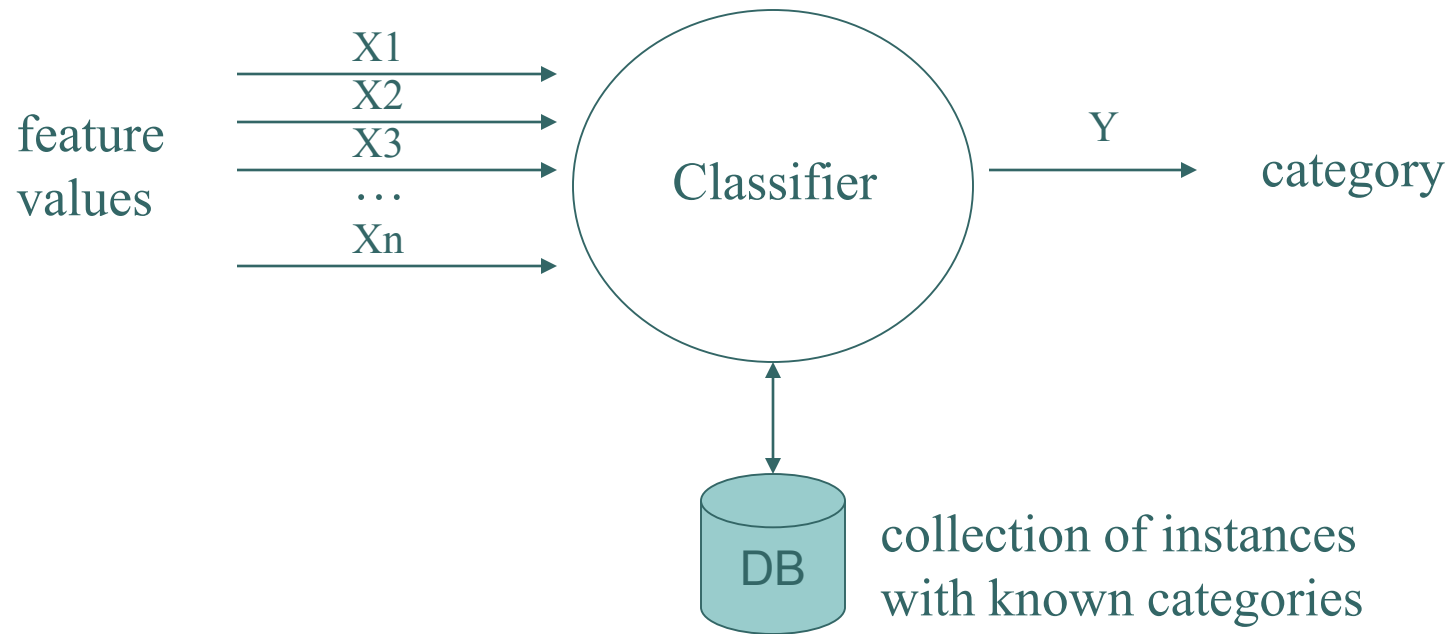




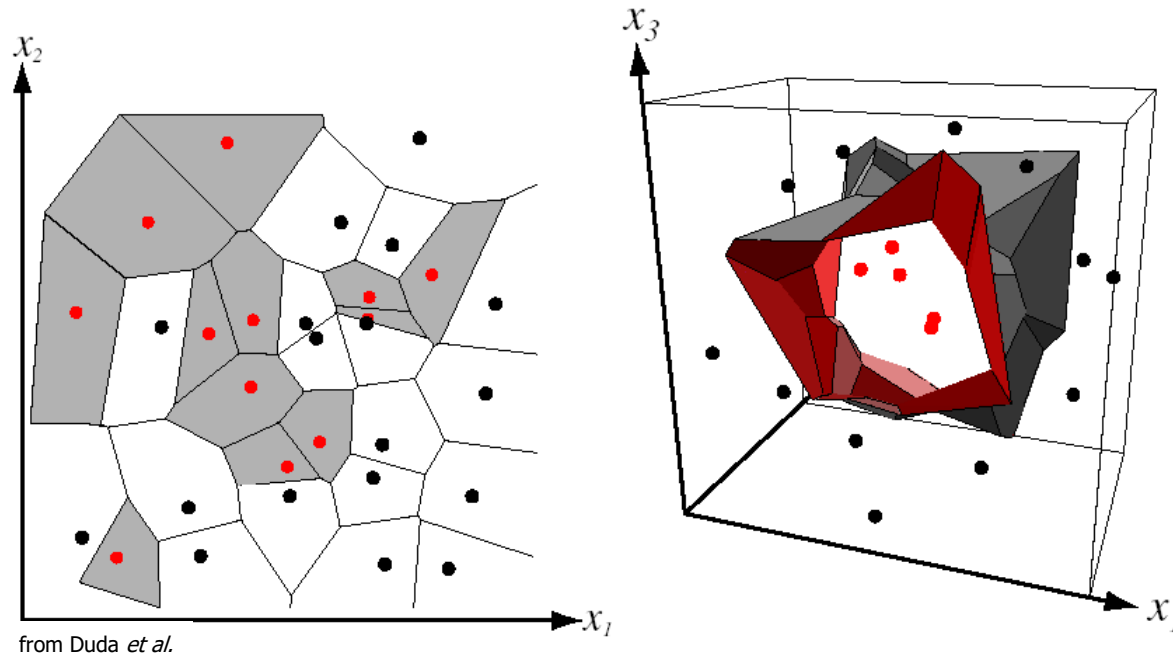
k Nearest Neighbors

Classifiers

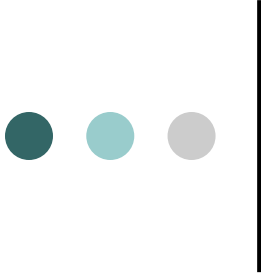


Nearest Neighbor Classifier

- Assign label of nearest training data point to each test data point



Voronoi partitioning of feature space
for two-category 2D and 3D data

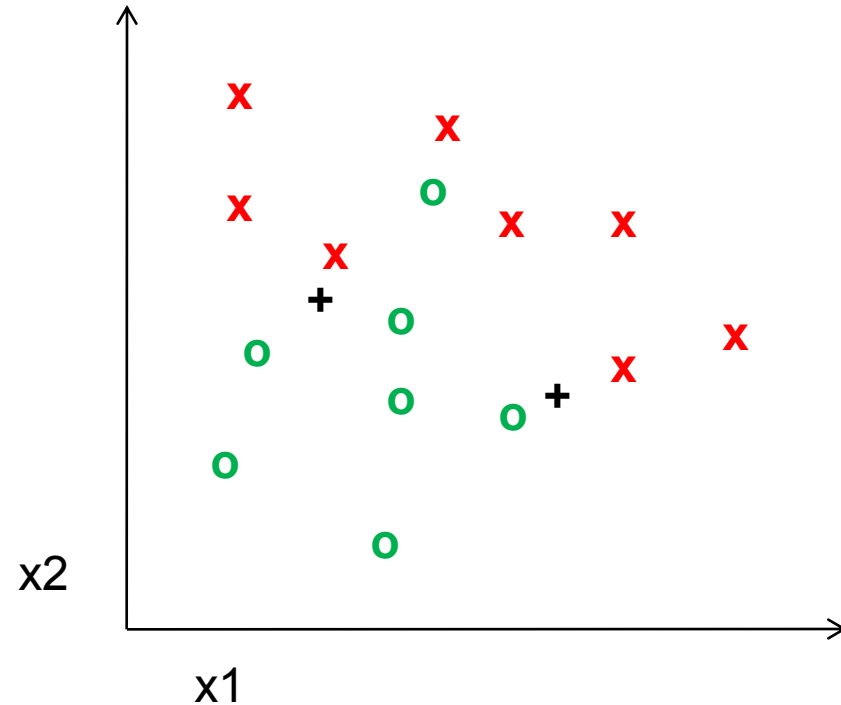


K - Nearest Neighbors

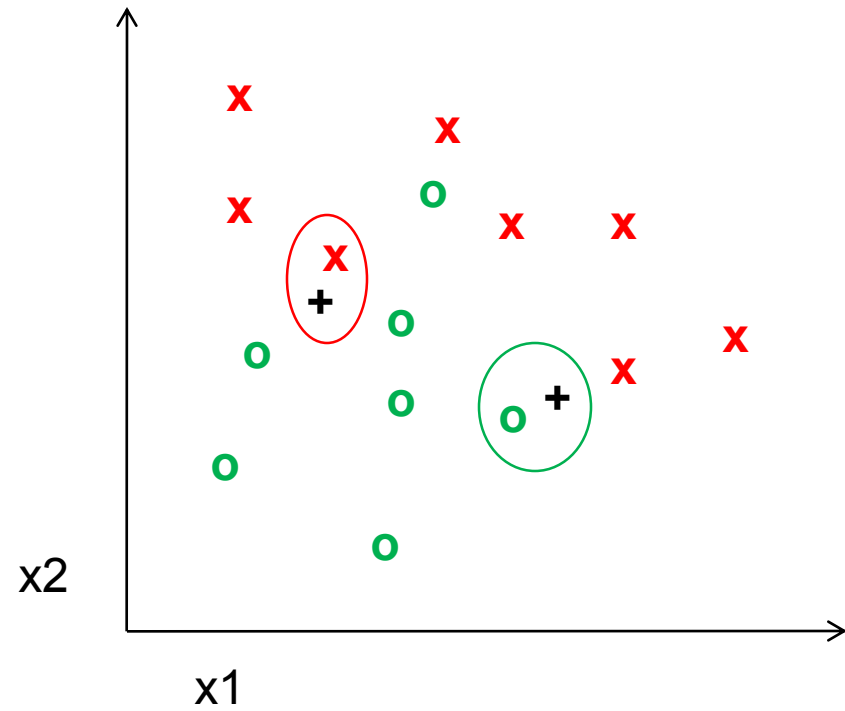
- For a given instance T , get the top k dataset instances that are “nearest” to T
 - Select a reasonable distance measure
- Inspect the category of these k instances, choose the category C that represent the most instances
- Conclude that T belongs to category C

K-nearest neighbor

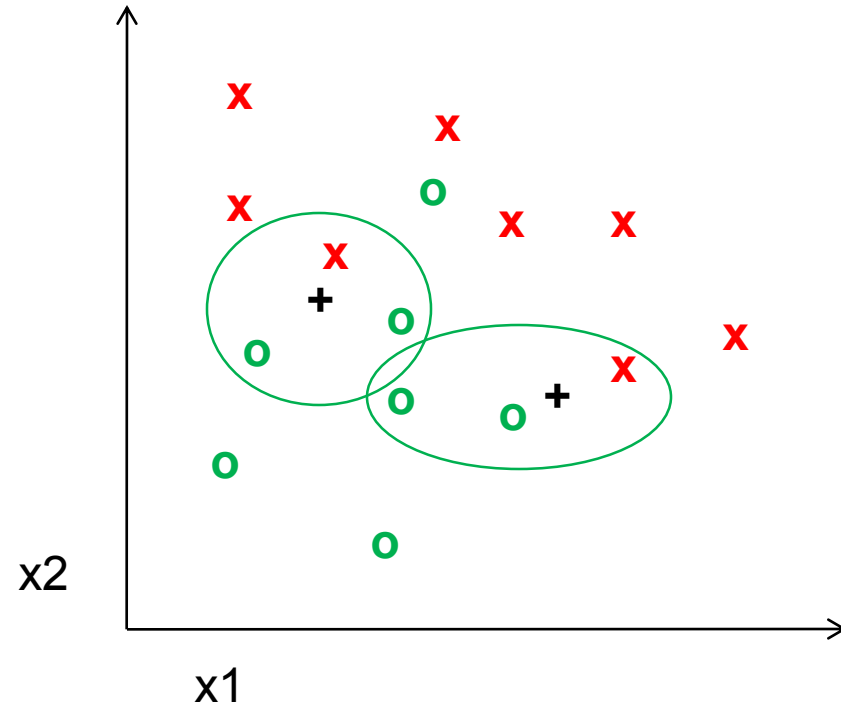
- Let '+' be the new data points, whose class is unknown.



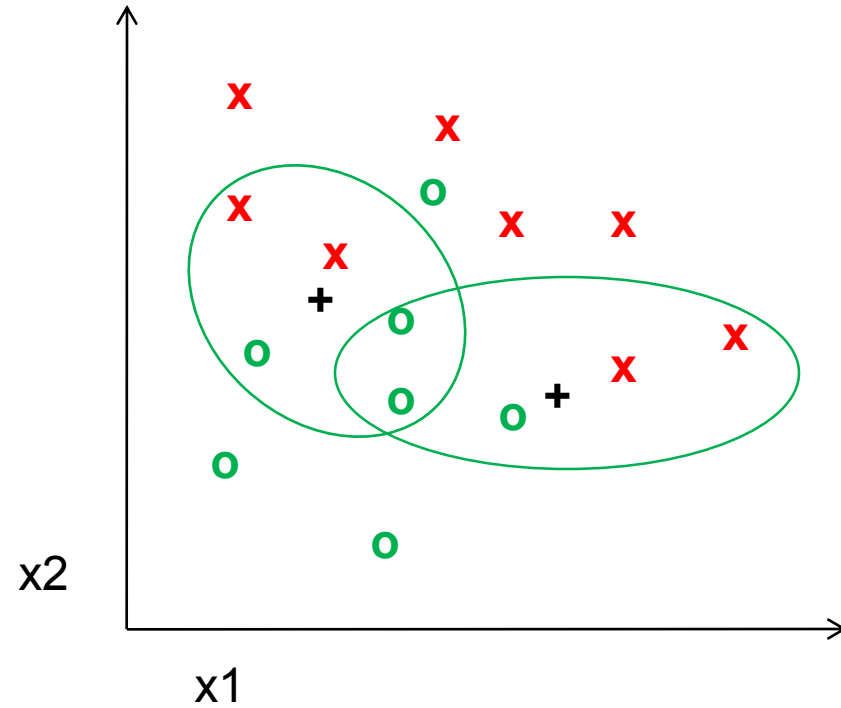
1-nearest neighbor



3-nearest neighbor



5-nearest neighbor



First: Nearest Neighbor (NN) Classifier

- Train
 - Remember all training images and their labels
- Predict
 - Find the closest (most similar) training image
 - Predict its label as the true label

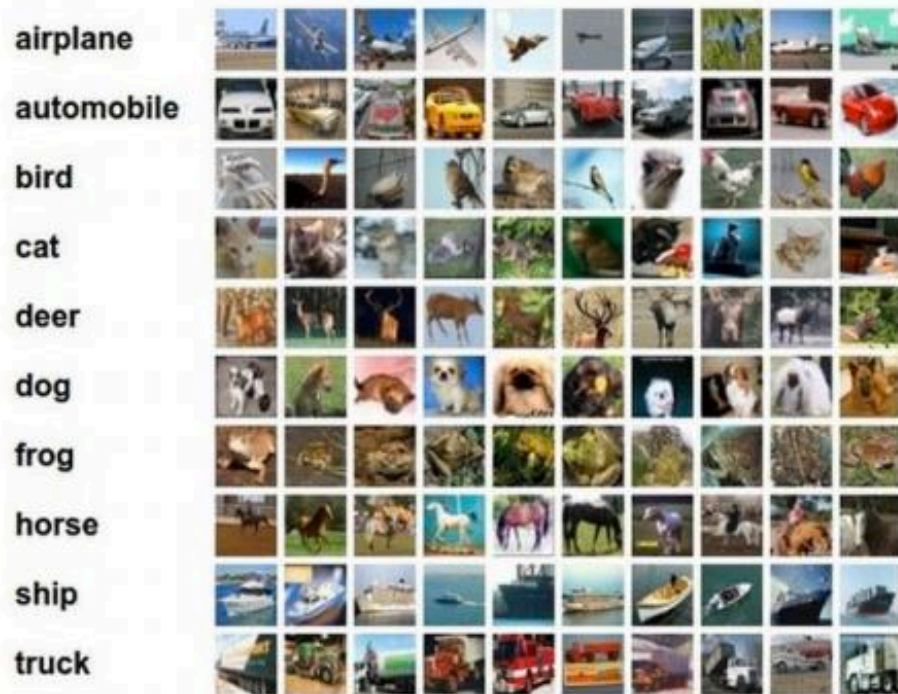
CIFAR-10 and NN results

Example dataset: **CIFAR-10**

10 labels

50,000 training images, each image is tiny: 32x32

10,000 test images.



CIFAR-10 and NN results

Example dataset: **CIFAR-10**

10 labels

50,000 training images

10,000 test images.



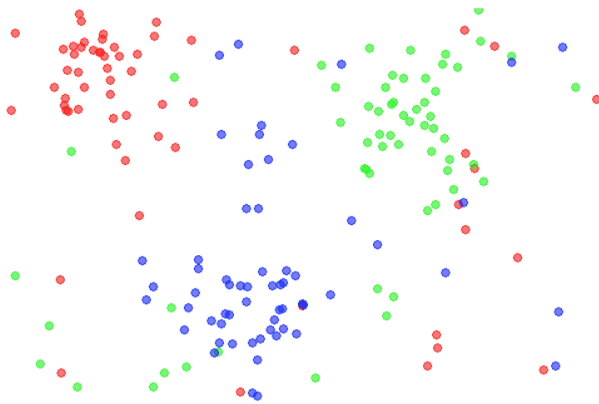
For every test image (first column),
examples of nearest neighbors in rows



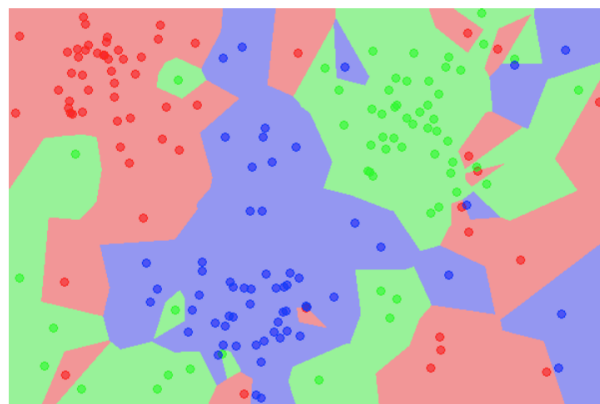
k-nearest neighbor

- Find the k closest points from training data
- Take **majority vote** from K closest points

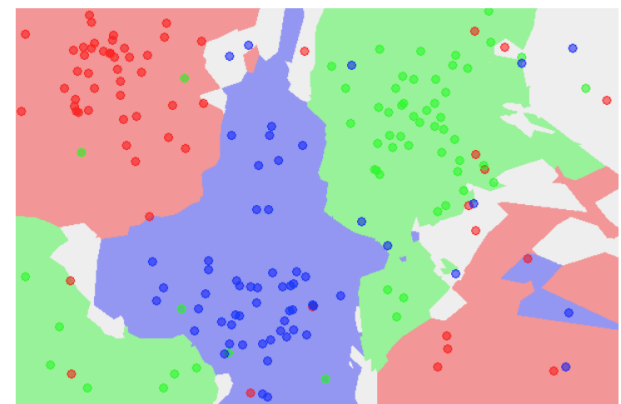
the data



NN classifier



5-NN classifier



What does this look like?



What does this look like?



How to find the most similar training image? What is the distance metric?

L1 distance:

$$d_1(I_1, I_2) = \sum_p |I_1^p - I_2^p|$$

Where I_1 denotes image 1,
and p denotes each pixel

test image

56	32	10	18
90	23	128	133
24	26	178	200
2	0	255	220

training image

10	20	24	17
8	10	89	100
12	16	178	170
4	32	233	112

-

=

pixel-wise absolute value differences

46	12	14	1
82	13	39	33
12	10	0	30
2	32	22	108

→ 456

Hyperparameters

- What is the **best distance** to use?
- What is the **best value of k** to use?
- These are **hyperparameters**: choices about the algorithm that we set rather than learn
- How do we set them?
 - One option: try them all and see what works best

Choice of distance metric

- Hyperparameter

L1 (Manhattan) distance

$$d_1(I_1, I_2) = \sum_p |I_1^p - I_2^p|$$

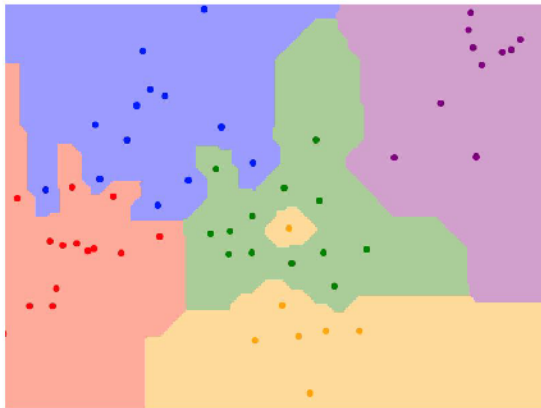
L2 (Euclidean) distance

$$d_2(I_1, I_2) = \sqrt{\sum_p (I_1^p - I_2^p)^2}$$

K-Nearest Neighbors: Distance Metric

L1 (Manhattan) distance

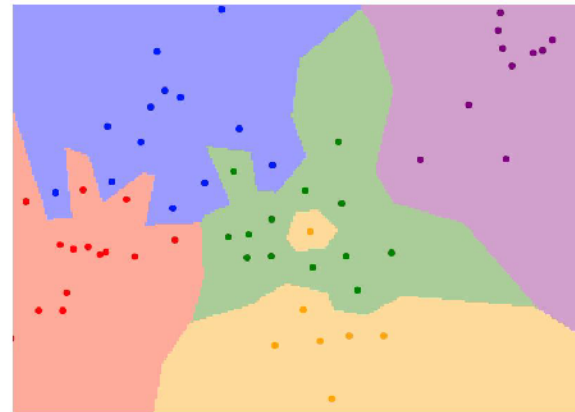
$$d_1(I_1, I_2) = \sum_p |I_1^p - I_2^p|$$



K = 1

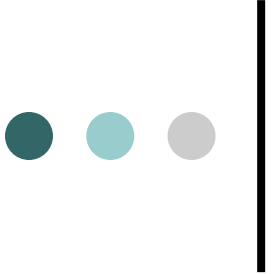
L2 (Euclidean) distance

$$d_2(I_1, I_2) = \sqrt{\sum_p (I_1^p - I_2^p)^2}$$



K = 1

Demo: <http://vision.stanford.edu/teaching/cs231n-demos/knn/>



Other distance measures

- City-block distance (Manhattan dist)
 - Add absolute value of differences
- Cosine similarity
 - Measure angle formed by the two samples (with the origin)
- Jaccard distance
 - Determine percentage of exact matches between the samples (not including unavailable data)
- Others

Distance Metrics

Minkowsky:

$$D(\mathbf{x}, \mathbf{y}) = \left(\sum_{i=1}^m |x_i - y_i|^r \right)^{1/r}$$

Euclidean:

$$D(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^m (x_i - y_i)^2}$$

Manhattan / city-block:

$$D(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^m |x_i - y_i|$$

Camberra:

$$D(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^m \frac{|x_i - y_i|}{|x_i + y_i|}$$

Chebychev:

$$D(\mathbf{x}, \mathbf{y}) = \max_{i=1}^m |x_i - y_i|$$

Quadratic:

$$D(\mathbf{x}, \mathbf{y}) = (\mathbf{x} - \mathbf{y})^T \mathbf{Q} (\mathbf{x} - \mathbf{y}) = \sum_{j=1}^m \left(\sum_{i=1}^m (x_i - y_i) q_{ji} \right) (x_j - y_j)$$

Q is a problem-specific positive
definite $m \times m$ weight matrix

Mahalanobis:

$$D(\mathbf{x}, \mathbf{y}) = [\det V]^{1/m} (\mathbf{x} - \mathbf{y})^T V^{-1} (\mathbf{x} - \mathbf{y})$$

V is the covariance matrix of $A_1..A_m$,
and A_j is the vector of values for
attribute j occurring in the training set
instances $1..n$.

Correlation:

$$D(\mathbf{x}, \mathbf{y}) = \frac{\sum_{i=1}^m (x_i - \bar{x}_i)(y_i - \bar{y}_i)}{\sqrt{\sum_{i=1}^m (x_i - \bar{x}_i)^2 \sum_{i=1}^m (y_i - \bar{y}_i)^2}}$$

$\bar{x}_i = \bar{y}_i$ and is the average value for
attribute i occurring in the training set.

Chi-square:

$$D(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^m \frac{1}{sum_i} \left(\frac{x_i}{size_x} - \frac{y_i}{size_y} \right)^2$$

sum_i is the sum of all values for attribute
 i occurring in the training set, and $size_x$ is
the sum of all values in the vector $\mathbf{x}$.

Kendall's Rank Correlation:

$$D(\mathbf{x}, \mathbf{y}) = 1 - \frac{2}{n(n-1)} \sum_{i=1}^m \sum_{j=1}^{i-1} \text{sign}(x_i - x_j) \text{sign}(y_i - y_j)$$

$\text{sign}(x) = -1, 0$ or 1 if $x < 0$,
 $x = 0$, or $x > 0$, respectively.

Figure 1. Equations of selected distance functions.
($\mathbf{x}$ and $\mathbf{y}$ are vectors of m attribute values).

● How to determine the good value for k ?

- Determined experimentally
- Start with $k=1$ and use a test set to validate the error rate of the classifier
- Repeat with $k=k+2$
- Choose the value of k for which the error rate is minimum
- Note: k should be odd number to avoid ties

Setting Hyperparameters

Idea #1: Choose hyperparameters
that work best on the data



Your Dataset

Setting Hyperparameters

Idea #1: Choose hyperparameters that work best on the data

BAD: $K = 1$ always works perfectly on training data



Your Dataset

Setting Hyperparameters

Idea #1: Choose hyperparameters that work best on the data

BAD: $K = 1$ always works perfectly on training data



Idea #2: Split data into **train** and **test**, choose hyperparameters that work best on test data



Setting Hyperparameters

Idea #1: Choose hyperparameters that work best on the data

BAD: $K = 1$ always works perfectly on training data



Idea #2: Split data into **train** and **test**, choose hyperparameters that work best on test data

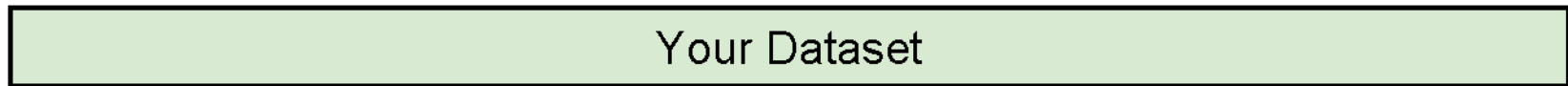
BAD: No idea how algorithm will perform on new data



Setting Hyperparameters

Idea #1: Choose hyperparameters that work best on the data

BAD: $K = 1$ always works perfectly on training data



Idea #2: Split data into **train** and **test**, choose hyperparameters that work best on test data

BAD: No idea how algorithm will perform on new data



Idea #3: Split data into **train**, **val**, and **test**; choose hyperparameters on val and evaluate on test

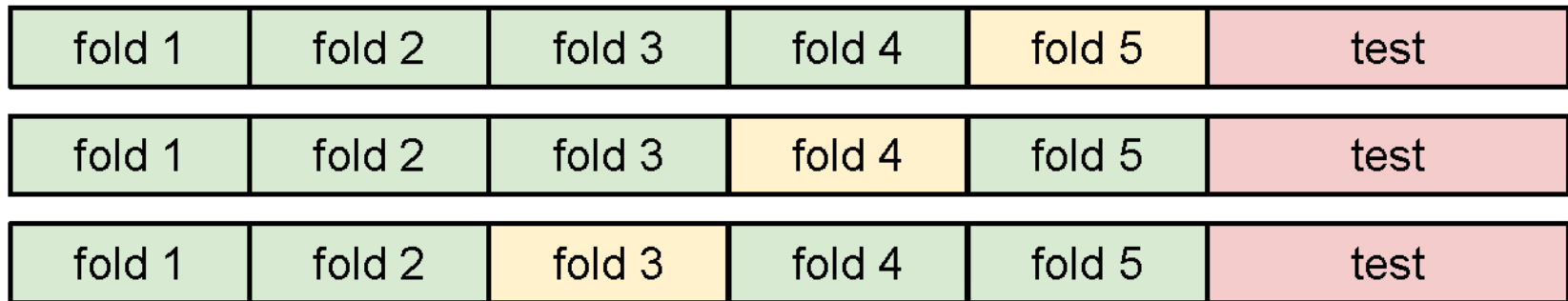
Better!



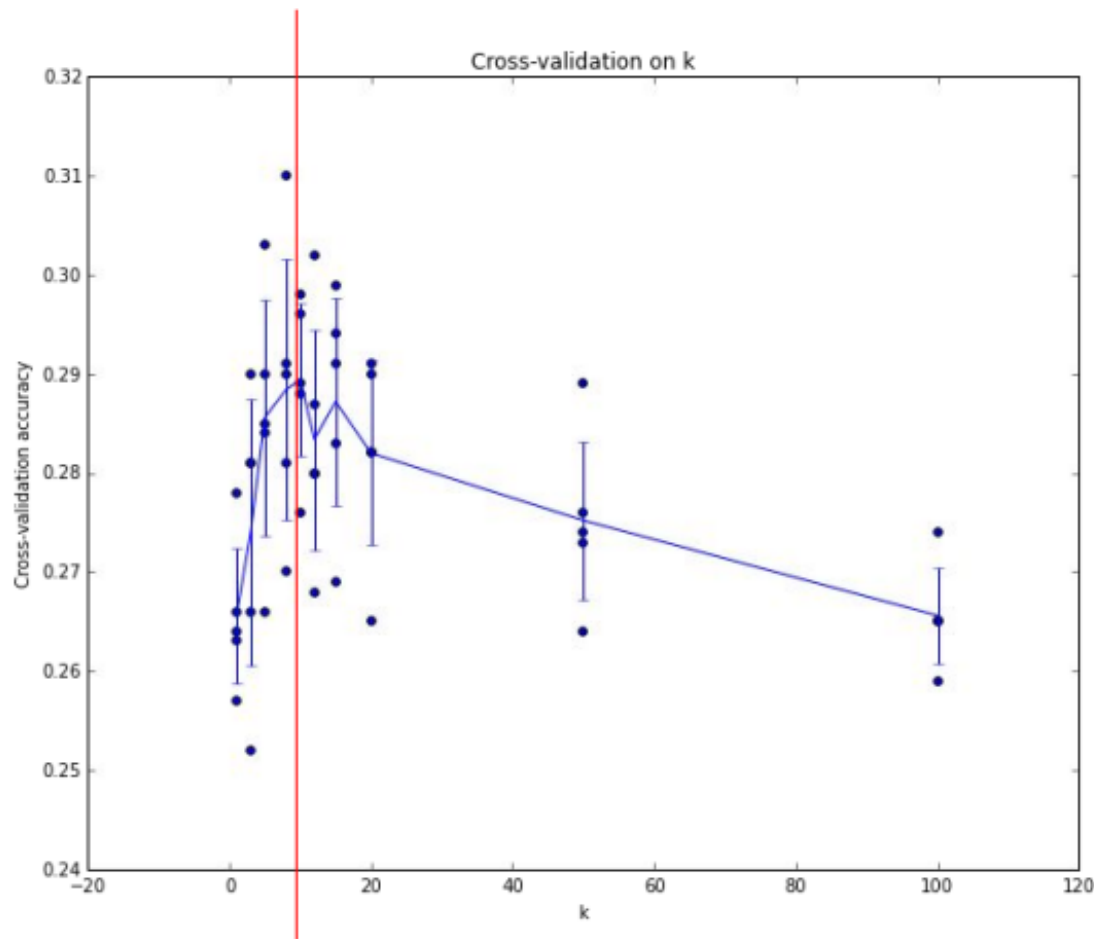
Setting Hyperparameters



Idea #4: Cross-Validation: Split data into **folds**,
try each fold as validation and average the results



Useful for small datasets, but not used too frequently in deep learning



Example of
5-fold cross-validation
for the value of **k**.

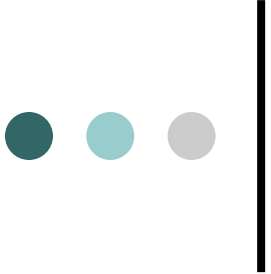
Each point: single
outcome.

The line goes
through the mean, bars
indicated standard
deviation

(Seems that $k \approx 7$ works best
for this data)

Recap: How to pick hyperparameters?

- Methodology
 - Train and test
 - Train, validate, test
- Train for original model
- Validate to find hyperparameters
- Test to understand generalizability

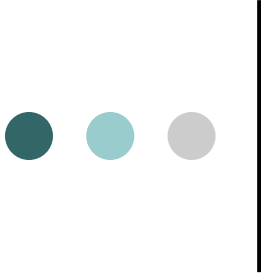


k-NN Time Complexity

- Suppose there are m instances and n features in the dataset
- Nearest neighbor algorithm requires computing m distances
- Each distance computation involves scanning through each feature value
- Running time complexity is proportional to $m \times n$

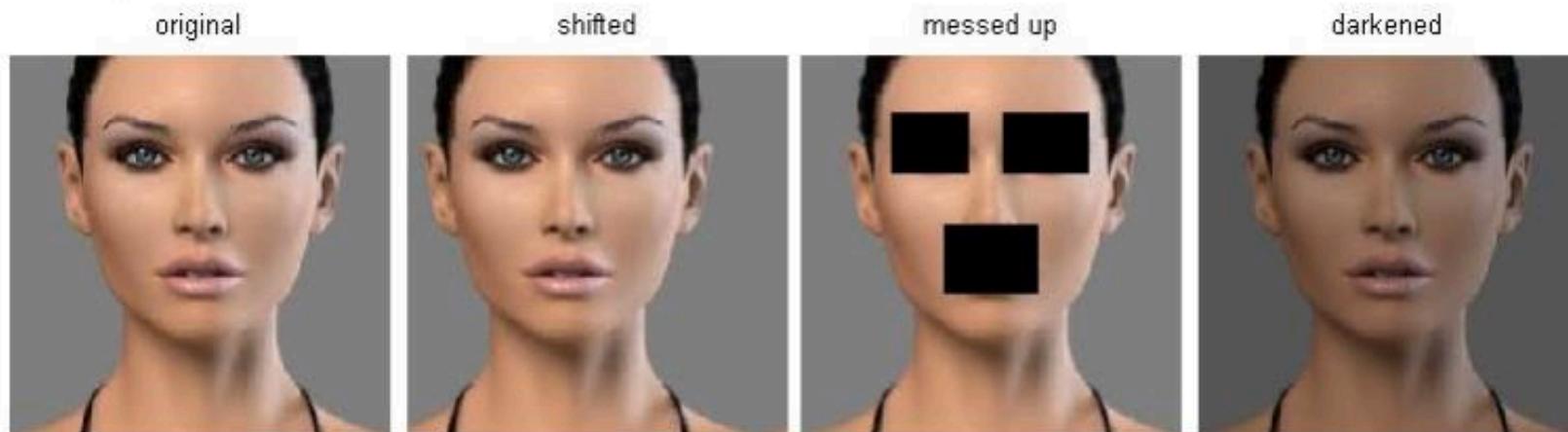
kNN -- Complexity and Storage

- N training images, M test images
- Training: $O(1)$
- Testing: $O(MN)$
- Hmm...
 - Normally need the opposite
 - Slow training (ok), fast testing (necessary)

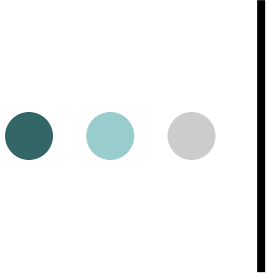
- 
- Disadvantage of kNN (instance-based methods) is that the costs of classifying new instances can be high
 - Nearly all computation takes place at classification time rather than learning time

k-Nearest Neighbor on images **never used**.

- terrible performance at test time
- distance metrics on level of whole images can be very unintuitive



(all 3 images have same L2 distance to the one on the left)



k-NN variations

- Value of k
 - Larger k increases confidence in prediction
 - Note that if k is too large, decision may be skewed
- Weighted evaluation of nearest neighbors
 - Plain majority may unfairly skew decision
 - Revise algorithm so that closer neighbors have greater “vote weight”
- Other distance measures

When to Consider Nearest Neighbors

- Instances map to points in R^d
- Less than 20 features (attributes) per instance, typically normalized
- Lots of training data

Advantages:

- Training is very fast
- Learn complex target functions
- Do not lose information

Disadvantages:

- Slow at query time
 - Presorting and indexing training samples into search trees reduces time
- Easily fooled by irrelevant features (attributes)

k-Nearest Neighbors: Summary

- In **image classification** we start with a **training set** of images and labels, and must predict labels on the **test set**
- The **K-Nearest Neighbors** classifier predicts labels based on nearest training examples
- Distance metric and K are **hyperparameters**
- Choose hyperparameters using the **validation set**; only run on the test set once at the very end!